T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CpE 010 - Data Structures and Algorithms	**SECTION:**	CpE21S1
DATE PERFORMED:	7-9-2026	**DATE SUBMITTED:**	7-13-2026
NAME:	Mauro, Ryan Lewis B. Mauro	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 1.1 [HOA1.1 Basic C++ Programming]

1. PROCEDURE OUTPUT

ILO A: Create Code That Follows the Basic C++ Code Structure

Section	Answer
Header File Declaration Section	`#include <iostream>`

Global Declaration Section	`void displaySum(int a, int b);`
	`bool isGreater(int a, int b);`
	`bool logicalOperations(bool x, bool y);`

Class Declaration and Method Definition Section	No class defined in this code

Main Function	`int main()`
	`{`
	`  int num1, num2;`
	`  int a, b;`
	`  bool value1, value2;`
	
	`  std::cout << "Enter first number: ";`
	`  std::cin >> num1;`
	
	`  std::cout << "Enter second number: ";`
	`  std::cin >> num2;`
	
	`  displaySum(num1, num2);`
	
	`  std::cout << "\nEnter value for A: ";`
	`  std::cin >> a;`
	
	`  std::cout << "Enter value for B: ";`
	`  std::cin >> b;`

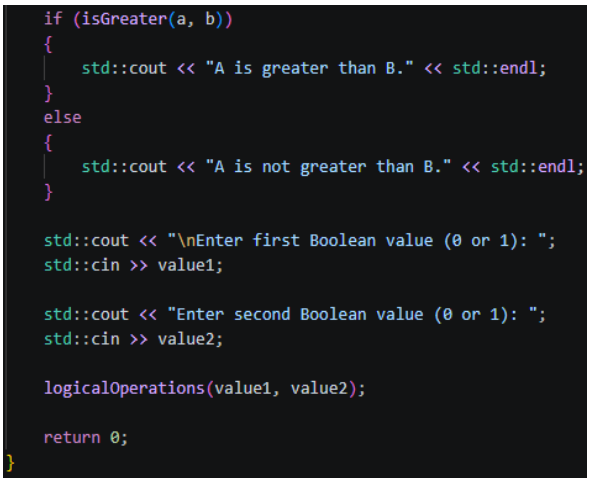
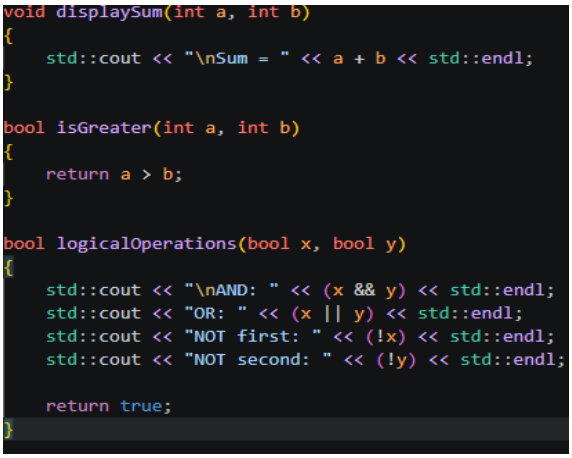
	
Method Definition	

Table 1-1. C++ Structure Code for Answer

```
    if (isGreater(a, b))
    {
        std::cout << "A is greater than B." << std::endl;
    }
    else
    {
        std::cout << "A is not greater than B." << std::endl;
    }

    std::cout << "\nEnter first Boolean value (0 or 1): ";
    std::cin >> value1;

    std::cout << "Enter second Boolean value (0 or 1): ";
    std::cin >> value2;

    logicalOperations(value1, value2);

    return 0;
}
```

```
void displaySum(int a, int b)
{
    std::cout << "\nSum = " << a + b << std::endl;
}

bool isGreater(int a, int b)
{
    return a > b;
}

bool logicalOperations(bool x, bool y)
{
    std::cout << "\nAND: " << (x && y) << std::endl;
    std::cout << "OR: " << (x || y) << std::endl;
    std::cout << "NOT first: " << (!x) << std::endl;
    std::cout << "NOT second: " << (!y) << std::endl;

    return true;
}
```

```
Enter first number: 12
Enter second number: 43

Sum = 55

Enter value for A: 12
Enter value for B: 2
A is greater than B.

Enter first Boolean value (0 or 1): 1
Enter second Boolean value (0 or 1): 0

AND: 0
OR: 1
NOT first: 0
NOT second: 1
```

The program follows the basic C++ program structure by using function prototypes before the main() function and placing the function definitions after it. It asks the user to enter values, making the program interactive and allowing different inputs each time it is run. The first function calculates and displays the sum of two numbers, while the second function checks if variable A is greater than variable B and returns the result. The third function accepts two Boolean values, displays the results of the logical operations (AND, OR, and NOT), and returns true to indicate that the function executed successfully. Each function has a specific purpose, making the code easier to read, understand, and maintain.The program meets all the activity requirements and demonstrates the proper use of function prototypes, function calls, user input, and basic C++ functions.

Table 1-2. ILO B output observations and comments.

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

Section	Answer
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	None

Class Declaration and Method Definition Section	`class Triangle`
	`{`
	`private:`
	`  double totalAngle, angleA, angleB, angleC;`
	
	`public:`
	`  Triangle(double A, double B, double C);`
	`  void setAngles(double A, double B, double C);`
	`  const bool validateTriangle();`
	`};`

Main Function

```cpp
int main()
{
    double A, B, C;

    std::cout << "Enter angle A: ";
    std::cin >> A;

    std::cout << "Enter angle B: ";
    std::cin >> B;

    std::cout << "Enter angle C: ";
    std::cin >> C;

    Triangle set1(A, B, C);

    if (set1.validateTriangle())
    {
        std::cout << "The shape is a valid triangle." << std::endl;
    }
    else
    {
        std::cout << "The shape is NOT a valid triangle." << std::endl;
    }

    return 0;
}
```

Method Definition

```cpp
Triangle::Triangle(double A, double B, double C)
{
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

void Triangle::setAngles(double A, double B, double C)
{
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

const bool Triangle::validateTriangle()
{
    return (totalAngle == 180);
}
```

Table 2-1. C++ Structure Code for Answer

```
Enter angle A: 40
Enter angle B: 30
Enter angle C: 110
The shape is a valid triangle.
PS C:\Users\Dell\Downloads\CPE21S1\
1Mauro\" ; if ($?) { g++ HOA1p1ILOB
Enter angle A: 60
Enter angle B: 60
Enter angle C: 30
The shape is NOT a valid triangle.
```

The program demonstrates the use of a class with private data members and public methods to organize related data and functions. It accepts three angle values from the user and stores them in a Triangle object using the constructor. The validateTriangle() method checks whether the sum of the three angles is equal to 180 degrees to determine if the shape is a valid triangle. This approach keeps the data protected while allowing controlled access through the class methods. The code is simple, easy to read, and shows how constructors, member functions, and objects work together in C++. The program meets the activity requirements by implementing an class definition, creating an instance of the class, and validating the user's input.

Table 2-2. ILO B output observations and comments.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. Create a C++ program to swap the two numbers in different variables.

```cpp
#include <iostream>

int main()
{
    int firstNumber, secondNumber, temp;

    std::cout << "Enter the first number: ";
    std::cin >> firstNumber;

    std::cout << "Enter the second number: ";
    std::cin >> secondNumber;

    std::cout << "\nBefore swapping:" << std::endl;
    std::cout << "First number: " << firstNumber << std::endl;
    std::cout << "Second number: " << secondNumber << std::endl;

    temp = firstNumber;
    firstNumber = secondNumber;
    secondNumber = temp;

    std::cout << "\nAfter swapping:" << std::endl;
    std::cout << "First number: " << firstNumber << std::endl;
    std::cout << "Second number: " << secondNumber << std::endl;

    return 0;
}
```

Figure 1.1<Code snippet of Supplementary no.1>

The program asks the user to enter two numbers and stores them in separate variables. It then uses a temporary variable to swap their values without losing either number. Finally, it displays the numbers before and after the swap to show that the values have been exchanged.

```
Enter the first number: 2
Enter the second number: 7

Before swapping:
First number: 2
Second number: 7

After swapping:
First number: 7
Second number: 2
```

Figure 1.2 <Output snippet of supplementary task no.1>

The output first shows the original values entered by the user. After the swapping process, the values are displayed again, but their positions are reversed. This confirms that the program successfully swapped the two numbers.

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

```cpp
#include <iostream>

double convertToFahrenheit(double kelvin);

int main()
{
    double kelvin;

    std::cout << "Enter temperature in Kelvin: ";
    std::cin >> kelvin;

    std::cout << "Temperature in Fahrenheit: "
              << convertToFahrenheit(kelvin) << std::endl;

    return 0;
}

double convertToFahrenheit(double kelvin)
{
    return (kelvin - 273.15) * 9 / 5 + 32;
}
```

Figure 2.1<Code snippet of Supplementary no.2>

The program asks the user to enter a temperature in Kelvin and passes the value to a function. The function converts the Kelvin temperature to Fahrenheit using the conversion formula and returns the result. The converted temperature is then displayed on the screen.

```
Enter temperature in Kelvin: 10.232
Temperature in Fahrenheit: -441.252
```

Figure 2.2 <Output snippet of supplementary task no.2>

The output shows the temperature entered by the user after it has been converted to Fahrenheit. For an input of 10.232 Kelvin, the program correctly displays -441.252°F. This confirms that the conversion function works as expected.

3. Create a C++ program that has a function that will calculate the distance between two points.

```cpp
#include <iostream>
#include <cmath>

double calculateDistance(double x1, double y1, double x2, double y2);

int main()
{
    double x1, y1, x2, y2;

    std::cout << "Enter x-coordinate of the first point: ";
    std::cin >> x1;

    std::cout << "Enter y-coordinate of the first point: ";
    std::cin >> y1;

    std::cout << "Enter x-coordinate of the second point: ";
    std::cin >> x2;

    std::cout << "Enter y-coordinate of the second point: ";
    std::cin >> y2;

    std::cout << "\nDistance between the two points: "
              << calculateDistance(x1, y1, x2, y2) << std::endl;

    return 0;
}

double calculateDistance(double x1, double y1, double x2, double y2)
{
    return std::sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
}
```

Figure 3.1<Code snippet of Supplementary no.3>

The program asks the user to enter the coordinates of two points. It passes these values to a function that calculates the distance using the distance formula and returns the result. The calculated distance is then displayed on the screen.

```
Enter x-coordinate of the first point: 12
Enter y-coordinate of the first point: 8
Enter x-coordinate of the second point: 4
Enter y-coordinate of the second point: 2

Distance between the two points: 10
```

Figure 3.2 <Output snippet of supplementary task no.3>

The output shows the distance between the two points entered by the user. In the sample, the distance between the points (12, 8) and (4, 2) is 10, which is the correct result. This confirms that the function performs the calculation correctly.

4. Modify the code given in ILO B and add the following functions:

a. A function to compute for the area of a triangle

b. A function to compute for the perimeter of a triangle

c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

```cpp
#include <iostream>
#include <cmath>

class Triangle
{
private:
    double totalAngle, angleA, angleB, angleC;
    double sideA, sideB, sideC;

public:
    Triangle(double A, double B, double C, double sA, double sB, double sC);
    void setAngles(double A, double B, double C);
    const bool validateTriangle();
    double getArea();
    double getPerimeter();
    void getTriangleType();
};

Triangle::Triangle(double A, double B, double C, double sA, double sB, double sC)
{
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;

    sideA = sA;
    sideB = sB;
    sideC = sC;
}
```

```cpp
void Triangle::setAngles(double A, double B, double C)
{
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

const bool Triangle::validateTriangle()
{
    return (totalAngle == 180);
}

double Triangle::getPerimeter()
{
    return sideA + sideB + sideC;
}

double Triangle::getArea()
{
    double s = getPerimeter() / 2;
    return std::sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
}

void Triangle::getTriangleType()
{
    if (angleA < 90 && angleB < 90 && angleC < 90)
    {
        std::cout << "Triangle Type: Acute-angled" << std::endl;
    }
```

```
    else if (angleA > 90 || angleB > 90 || angleC > 90)
    {
        std::cout << "Triangle Type: Obtuse-angled" << std::endl;
    }
    else
    {
        std::cout << "Triangle Type: Others" << std::endl;
    }
}

int main()
{
    double A, B, C;
    double sideA, sideB, sideC;

    std::cout << "Enter angle A: ";
    std::cin >> A;

    std::cout << "Enter angle B: ";
    std::cin >> B;
    std::cout << "Enter angle C: ";
    std::cin >> C;

    std::cout << "Enter side A: ";
    std::cin >> sideA;

    std::cout << "Enter side B: ";
    std::cin >> sideB;

    std::cout << "Enter side C: ";
    std::cin >> sideC;

    Triangle set1(A, B, C, sideA, sideB, sideC);

    if (set1.validateTriangle())
    {
        std::cout << "The shape is a valid triangle." << std::endl;
        std::cout << "Perimeter: " << set1.getPerimeter() << std::endl;
        std::cout << "Area: " << set1.getArea() << std::endl;
        set1.getTriangleType();
    }
    else
    {
        std::cout << "The shape is NOT a valid triangle." << std::endl;
    }

    return 0;
}
```

Figure 4.1<Code snippet of Supplementary no.4>

The program extends the original Triangle class by adding functions to compute the area, perimeter, and type of the triangle. It accepts both the triangle's angles and side lengths from the user, validates the triangle, and then performs the requested calculations. Each task is handled by a separate member function, making the code organized and easy to understand.

```
Enter angle A: 40
Enter angle B: 30
Enter angle C: 110
Enter side A: 5
Enter side B: 10
Enter side C: 15
The shape is a valid triangle.
Perimeter: 30
Area: 0
Triangle Type: Obtuse-angled
```

Figure 4.2 <Output snippet of supplementary task no.4>

The output first checks whether the entered angles form a valid triangle by confirming that their sum is 180 degrees. If the triangle is valid, the program displays its perimeter, area, and whether it is acute-angled, obtuse-angled, or another type. This verifies that the added functions work correctly using the user's input.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, code assistance (How to use a private and public class), and Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **For grammar correction and I had to ask for ideas on how to use the private and public class since I'm not that familiar yet.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.